

Figure 11.14 Anchor-block and overflow-block structures.

The file structures of Figures 11.9 and 11.13 both use pointers; the difference is that, in Figure 11.9, we use pointers to chain together only deleted records, whereas in Figure 11.13, we chain together all records pertaining to the same branch.

A disadvantage to the structure of Figure 11.13 is that we waste space in all records except the first in a chain. The first record needs to have the *branch-name* value, but subsequent records do not. Nevertheless, we need to include a field for *branch-name* in all records, lest the records not be of fixed length. This wasted space is significant, since we expect, in practice, that each branch has a large number of accounts. To deal with this problem, we allow two kinds of blocks in our file:

1. **Anchor block**, which contains the first record of a chain
2. **Overflow block**, which contains records other than those that are the first record of a chain

Thus, all records *within a block* have the same length, even though not all records in the file have the same length. Figure 11.14 shows this file structure.

11.7 Organization of Records in Files

So far, we have studied how records are represented in a file structure. An instance of a relation is a set of records. Given a set of records, the next question is how to organize them in a file. Several of the possible ways of organizing records in files are:

- **Heap file organization**. Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is a single file for each relation
- **Sequential file organization**. Records are stored in sequential order, according to the value of a "search key" of each record. Section 11.7.1 describes this organization.

- **Hashing file organization**. A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed. Chapter 12 describes this organization; it is closely related to the indexing structures described in that chapter.

Generally, a separate file is used to store the records of each relation. However, in a **clustering file organization**, records of several different relations are stored in the same file; further, related records of the different relations are stored on the same block, so that one I/O operation fetches related records from all the relations. For example, records of the two relations can be considered to be related if they would match in a join of the two relations. Section 11.7.2 describes this organization.

11.7.1 Sequential File Organization

A **sequential file** is designed for efficient processing of records in sorted order based on some search-key. A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey. To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order. Furthermore, to minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

Figure 11.15 shows a sequential file of *account* records taken from our banking example. In that example, the records are stored in search-key order, using *branch-name* as the search key.

The sequential file organization allows records to be read in sorted order; that can be useful for display purposes, as well as for certain query-processing algorithms that we shall study in Chapter 13.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single



Figure 11.15 Sequential file for *account* records.

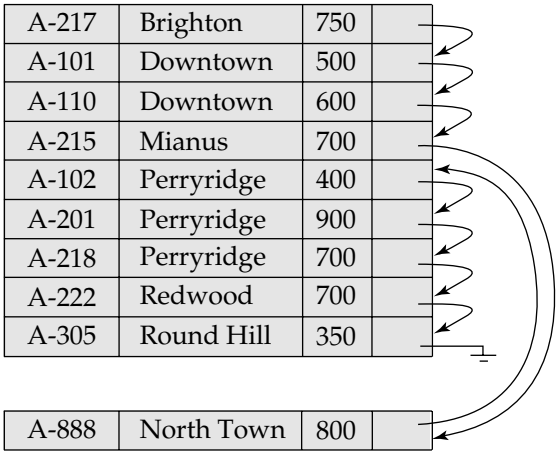


Figure 11.16 Sequential file after an insertion.

insertion or deletion. We can manage deletion by using pointer chains, as we saw previously. For insertion, we apply the following rules:

1. Locate the record in the file that comes before the record to be inserted in search-key order.
2. If there is a free record (that is, space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new record in an *overflow block*. In either case, adjust the pointers so as to chain together the records in search-key order.

Figure 11.16 shows the file of Figure 11.15 after the insertion of the record (North Town, A-888, 800). The structure in Figure 11.16 allows fast insertion of new records, but forces sequential file-processing applications to process records in an order that does not match the physical order of the records.

If relatively few records need to be stored in overflow blocks, this approach works well. Eventually, however, the correspondence between search-key order and physical order may be totally lost, in which case sequential processing will become much less efficient. At this point, the file should be **reorganized** so that it is once again physically in sequential order. Such reorganizations are costly, and must be done during times when the system load is low. The frequency with which reorganizations are needed depends on the frequency of insertion of new records. In the extreme case in which insertions rarely occur, it is possible always to keep the file in physically sorted order. In such a case, the pointer field in Figure 11.15 is not needed.

11.7.2 Clustering File Organization

Many relational-database systems store each relation in a separate file, so that they can take full advantage of the file system that the operating system provides. Usually, tuples of a relation can be represented as fixed-length records. Thus, relations

can be mapped to a simple file structure. This simple implementation of a relational database system is well suited to low-cost database implementations as in, for example, embedded systems or portable devices. In such systems, the size of the database is small, so little is gained from a sophisticated file structure. Furthermore, in such environments, it is essential that the overall size of the object code for the database system be small. A simple file structure reduces the amount of code needed to implement the system.

This simple approach to relational-database implementation becomes less satisfactory as the size of the database increases. We have seen that there are performance advantages to be gained from careful assignment of records to blocks, and from careful organization of the blocks themselves. Clearly, a more complicated file structure may be beneficial, even if we retain the strategy of storing each relation in a separate file.

However, many large-scale database systems do not rely directly on the underlying operating system for file management. Instead, one large operating-system file is allocated to the database system. The database system stores all relations in this one file, and manages the file itself. To see the advantage of storing many relations in one file, consider the following SQL query for the bank database:

select *account-number, customer-name, customer-street, customer-city*
from *depositor, customer*
where *depositor.customer-name* = *customer.customer-name*

This query computes a join of the *depositor* and *customer* relations. Thus, for each tuple of *depositor*, the system must locate the *customer* tuples with the same value for *customer-name*. Ideally, these records will be located with the help of *indices*, which we shall discuss in Chapter 12. Regardless of how these records are located, however, they need to be transferred from disk into main memory. In the worst case, each record will reside on a different block, forcing us to do one block read for each record required by the query.

As a concrete example, consider the *depositor* and *customer* relations of Figures 11.17 and 11.18, respectively. In Figure 11.19, we show a file structure designed for efficient execution of queries involving $depositor \bowtie customer$. The *depositor* tuples for each *customer-name* are stored near the *customer* tuple for the corresponding *customer-name*. This structure mixes together tuples of two relations, but allows for efficient processing of the join. When a tuple of the *customer* relation is read, the entire block containing that tuple is copied from disk into main memory. Since the corresponding

customer-name	*account-number*
Hayes	A-102
Hayes	A-220
Hayes	A-503
Turner	A-305

Figure 11.17 The *depositor* relation.

customer-name	*customer-street*	*customer-city*
Hayes	Main	Brooklyn
Turner	Putnam	Stamford

Figure 11.18 The *customer* relation.

depositor tuples are stored on the disk near the *customer* tuple, the block containing the *customer* tuple contains tuples of the *depositor* relation needed to process the query. If a customer has so many accounts that the *depositor* records do not fit in one block, the remaining records appear on nearby blocks.

A **clustering file organization** is a file organization, such as that illustrated in Figure 11.19 that stores related records of two or more relations in each block. Such a file organization allows us to read records that would satisfy the join condition by using one block read. Thus, we are able to process this particular query more efficiently.

Our use of clustering has enhanced processing of a particular join (*depositor* ⋈ *customer*), but it results in slowing processing of other types of query. For example,

select *
from *customer*

requires more block accesses than it did in the scheme under which we stored each relation in a separate file. Instead of several *customer* records appearing in one block, each record is located in a distinct block. Indeed, simply finding all the *customer* records is not possible without some additional structure. To locate all tuples of the *customer* relation in the structure of Figure 11.19, we need to chain together all the records of that relation using pointers, as in Figure 11.20.

When clustering is to be used depends on the types of query that the database designer believes to be most frequent. Careful use of clustering can produce significant performance gains in query processing.

11.8 Data-Dictionary Storage

So far, we have considered only the representation of the relations themselves. A relational-database system needs to maintain data *about* the relations, such as the

Hayes	Main	Brooklyn
Hayes	A-102	
Hayes	A-220	
Hayes	A-503	
Turner	Putnam	Stamford
Turner	A-305	

Figure 11.19 Clustering file structure.

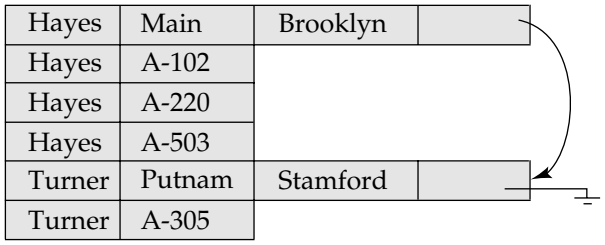


Figure 11.20 Clustering file structure with pointer chains.

schema of the relations. This information is called the **data dictionary**, or **system catalog**. Among the types of information that the system must store are these:

- Names of the relations
- Names of the attributes of each relation
- Domains and lengths of attributes
- Names of views defined on the database, and definitions of those views
- Integrity constraints (for example, key constraints)

In addition, many systems keep the following data on users of the system:

- Names of authorized users
- Accounting information about users
- Passwords or other information used to authenticate users

Further, the database may store statistical and descriptive data about the relations, such as:

- Number of tuples in each relation
- Method of storage for each relation (for example, clustered or nonclustered)

The data dictionary may also note the storage organization (sequential, hash or heap) of relations, and the location where each relation is stored:

- If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
- If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.

In Chapter 12, in which we study indices, we shall see a need to store information about each index on each of the relations:

- Name of the index
- Name of the relation being indexed
- Attributes on which the index is defined
- Type of index formed

All this information constitutes, in effect, a miniature database. Some database systems store this information by using special-purpose data structures and code. It is generally preferable to store the data about the database in the database itself. By using the database to store system data, we simplify the overall structure of the system and harness the full power of the database for fast access to system data.

The exact choice of how to represent system data by relations must be made by the system designers. One possible representation, with primary keys underlined, is

Relation-metadata (*relation-name*, *number-of-attributes*, *storage-organization*, *location*)
Attribute-metadata (*attribute-name*, *relation-name*, *domain-type*, *position*, *length*)
User-metadata (*user-name*, *encrypted-password*, *group*)
Index-metadata (*index-name*, *relation-name*, *index-type*, *index-attributes*)
View-metadata (*view-name*, *definition*)

In this representation, the attribute *index-attributes* of the relation *Index-metadata* is assumed to contain a list of one or more attributes, which can be represented by a character string such as "*branch-name, branch-city*". The *Index-metadata* relation is thus not in first normal form; it can be normalized, but the above representation is likely to be more efficient to access. The data dictionary is often stored in a non-normalized form to achieve fast access.

The storage organization and location of the *Relation-metadata* itself must be recorded elsewhere (for example, in the database code itself), since we need this information to find the contents of *Relation-metadata*.

11.9 Storage for Object-Oriented Databases**

The file-organization techniques described in Section 11.7—the heap, sequential, hashing and clustering organizations—can also be used for storing objects in an object-oriented database. However, some extra features are needed to support object-oriented database features, such as set-valued fields and persistent pointers.

11.9.1 Mapping of Objects to Files

The mapping of objects to files is in many ways like the mapping of tuples to files in a relational system. At the lowest level of data representation, both tuples and the data parts of objects are simply sequences of bytes. We can therefore store object data in the file structures described in this chapter, with some modifications which we note next.

Objects in object-oriented databases may lack the uniformity of tuples in relational databases. For example, fields of records may be sets; in relational databases, in con-

trast, data are typically required to be (at least) in first normal form. Furthermore, objects may be extremely large. Such objects have to be managed differently from records in a relational system.

We can implement set-valued fields that have a small number of elements using data structures such as linked lists. Set-valued fields that have a larger number of elements can be implemented as relations in the database. Set-valued fields of objects can also be eliminated at the *storage level* by normalization: A relation is created containing one tuple for each value of a set-valued field of an object. Each tuple also contains the object identifier of the object. However, this relation is not made visible to the upper levels of the database system. The storage system gives the upper levels of the database system the view of a set-valued field, even though the set-valued field has actually been normalized by creating a new relation.

Some applications include extremely large objects that are not easily decomposed into smaller components. Such large objects may each be stored in a separate file. We discuss this idea further in Section 11.9.6.

11.9.2 Implementation of Object Identifiers

Since objects are identified by object identifiers (OIDs), an object-storage system needs a mechanism to locate an object, given an OID. If the OIDs are **logical OID**s—that is, they do not specify the location of the object—then the storage system must maintain an index that maps OIDs to the actual location of the object. If the OIDs are **physical OID**s—that is, they encode the location of the object—then the object can be found directly. Physical OIDs typically have the following three parts:

1. A volume or file identifier
2. A block identifier within the volume or file
3. An offset within the block

A volume is a logical unit of storage that usually corresponds to a disk.

In addition, physical OIDs may contain a **unique identifier**, which is an integer that distinguishes the OID from the identifiers of other objects that happened to be stored at the same location earlier, and were deleted or moved elsewhere. The unique identifier is also stored with the object, and the identifiers in an OID and the corresponding object should match. If the unique identifier in a physical OID does not match the unique identifier in the object to which that OID points, the system detects that the pointer is a dangling pointer, and signals an error. (A **dangling pointer** is a pointer that does not point to a valid object.) Figure 11.21 illustrates this scheme.

Such pointer errors occur when physical OIDs corresponding to old objects that have been deleted are used accidentally. If the space occupied by the object had been reallocated, there may be a new object in the location, and it may get incorrectly addressed by the identifier of the old object. If a dangling pointer is not detected, it could cause corruption of a new object stored at the same location. The unique identifier helps to detect such errors, since the unique identifiers of the old physical OID and the new object will not match.

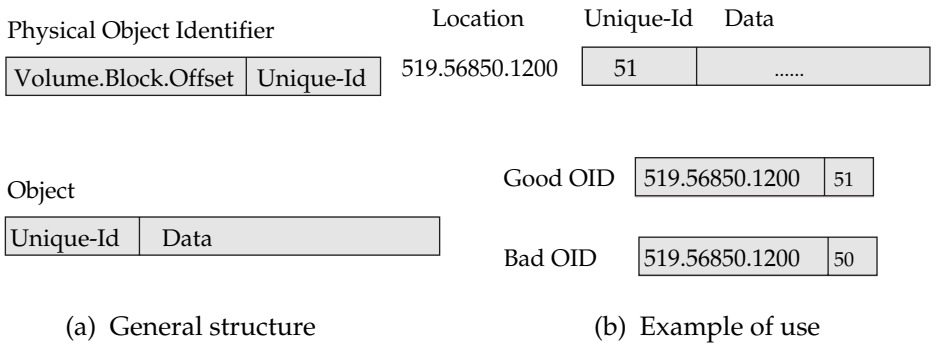


Figure 11.21 Unique identifiers in an OID.

Suppose that an object has to be moved to a new block, perhaps because the size of the object has increased, and the old block has no extra space. Then, the physical OID will point to the old block, which no longer contains the object. Rather than change the OID of the object (which involves changing every object that points to this one), we leave behind a **forwarding address** at the old location. When the database tries to locate the object, it finds the forwarding address instead of the object; it then uses the forwarding address to locate the object.

11.9.3 Management of Persistent Pointers

We implement persistent pointers in a persistent programming language by using OIDs. In some implementations, persistent pointers are physical OIDs; in others, they are logical OIDs. An important difference between persistent pointers and in-memory pointers is the size of the pointer. In-memory pointers need to be only big enough to address all virtual memory. On most current computers, in-memory pointers are usually 4 bytes long, which is sufficient to address 4 gigabytes of memory. The most recent computer architectures have pointers that are 8 bytes long,

Persistent pointers need to address all the data in a database. Since database systems are often bigger than 4 gigabytes, persistent pointers are usually at least 8 bytes long. Many object-oriented databases also provide unique identifiers in persistent pointers, to catch dangling references. This feature further increases the size of persistent pointers. Thus, persistent pointers may be substantially longer than in-memory pointers.

The action of looking up an object, given its identifier, is called **dereferencing**. Given an in-memory pointer (as in C++), looking up the object is merely a memory reference. Given a persistent pointer, dereferencing an object has an extra step—finding the actual location of the object in memory by looking up the persistent pointer in a table. If the object is not already in memory, it has to be loaded from disk. We can implement the table lookup fairly efficiently by using a hash table data structure, but the lookup is still slow compared to a pointer dereference, even if the object is already in memory.

Pointer swizzling is a way to cut down the cost of locating persistent objects that are already present in memory. The idea is that, when a persistent pointer is first dereferenced, the system locates the object and brings it into memory if it is not already there. Now the system carries out an extra step—it stores an in-memory pointer to the object in place of the persistent pointer. The next time that the *same* persistent pointer is dereferenced, the in-memory location can be read out directly, so the costs of locating the object are avoided. (When persistent objects have to be moved from memory back to disk to make space for other persistent objects, the system must carry out an extra step to ensure that the object is still in memory. Correspondingly, when an object is written out, any persistent pointers that it contained and that were swizzled have to be **deswizzled**, that is, converted back to their persistent representation. Pointer swizzling on pointer dereference, as described here, is called **software swizzling**.

Buffer management is more complicated if pointer swizzling is used, since the physical location of an object must not change once that object is brought into the buffer. One way to ensure that it will not change is to pin pages containing swizzled objects in the buffer pool, so that they are never replaced until the program that performed the swizzling has finished execution. See the bibliographical notes for more complex buffer-management schemes, based on virtual-memory mapping techniques, that make it unnecessary to pin the buffer pages.

11.9.4 Hardware Swizzling

Having two types of pointers, persistent and transient (in-memory), is inconvenient. Programmers have to remember the type of the pointers, and may have to write code twice—once for the persistent pointers and once for the in-memory pointers. It would be simpler if both persistent and in-memory pointers were of the same type.

A simple way to merge persistent and in-memory pointer types is just to extend the length of in-memory pointers to the same size as persistent pointers, and to use 1 bit of the identifier to distinguish between persistent and in-memory pointers. However, the storage cost of longer persistent pointers will have to be borne by in-memory pointers as well; understandably, this scheme is unpopular.

We shall describe a technique called **hardware swizzling**, which uses the virtual-memory-management hardware present in most current computer systems to address this problem. When data in a virtual memory page are accessed, and the operating system detects that the page does not have real storage allocated for it, or has been access protected, then a **segmentation violation** is said to occur.[3] Many operating systems provide a mechanism to specify a function to be called when a segmentation violation occurs, and a mechanism to allocate storage for a page in virtual address space, and to set that page's access permissions. In most Unix systems, the `mmap` system call provides this latter functionality. Hardware swizzling makes clever use of the above mechanisms.

3. The term **page fault** is sometimes used instead of *segmentation violation*, although access protection violations are generally not considered to be page faults.

Hardware swizzling has two major advantages over software swizzling:

1. It is able to store persistent pointers in objects in the same amount of space as in-memory pointers require (along with extra storage external to the object).
2. It transparently converts between persistent pointers and in-memory pointers in a clever and efficient way. Software written to deal with in-memory pointers can thereby deal with persistent pointers as well, without any changes.

11.9.4.1 Pointer Representation

Hardware swizzling uses the following representation of persistent pointers contained in objects that are on disk. A persistent pointer is conceptually split into two parts: a page identifier in the database, and an offset within the page.[4] The page identifier in a persistent pointer is actually a small indirect pointer, which we call the short page identifier. Each page (or other unit of storage) has a translation table that provides a mapping from the short page identifiers to full database page identifiers. The system has to look up the short page identifier in a persistent pointer in the translation table to find the full page identifier.

The translation table, in the worst case, will be only as big as the maximum number of pointers that can be contained in objects in a page; with a page size of 4096, and a pointer size of 4 bytes, the maximum number of pointers is 1024. In practice, the translation table is likely to contain much less than the maximum number of elements (1024 in our example) and will not consume excessive space. The short page identifier needs to have only enough bits to identify a row in the table; with a maximum table size of 1024, only 10 bits are required. Hence, a small number of bits is enough to store the short page identifier. Thus, the translation table permits an entire persistent pointer to fit into the same space as an in-memory pointer. Even though only a few bits are needed for the short page identifier, all the bits of an in-memory pointer, other than the page-offset bits, are used as the short page identifier. This architecture facilitates swizzling, as we shall see.

The persistent-pointer representation scheme appears in Figure 11.22, where there are three objects in the page, each containing a persistent pointer. The translation table gives the mapping between short page identifiers and the full database page identifiers for each of the short page identifiers in these persistent pointers. The database page identifiers are shown in the format *volume.page.offset*.

Each page maintains extra information so that all persistent pointers in the page can be found. The system updates the information when an object is created or deleted in the page. The need to locate all the persistent pointers in a page will become clear later.

4. The term **page** is generally used to refer to a real-memory or virtual-memory page, and the term **block** is used to refer to disk blocks in the database. In hardware swizzling, these have to be of the same size, and database blocks are fetched into virtual memory pages. We shall use the terms *page* and *block* interchangeably in this section.

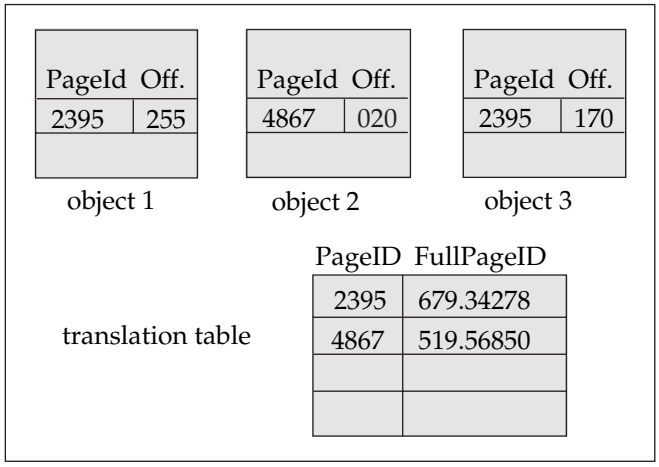


Figure 11.22 Page image before swizzling.

11.9.4.2 Swizzling Pointers on a Page

Initially no page of the database has been allocated a page in virtual memory. Virtual-memory pages may be allocated to database pages even before they are actually loaded, as we will see shortly. Database pages get loaded into virtual-memory when the database system needs to access data on the page. Before a database page is loaded, the system allocates a virtual-memory page to the database page if one has not already been allocated. The system then loads the database page into the virtual-memory page it has allocated to it.

When the system loads a database page P into virtual memory, it does pointer swizzling on the page: It locates all persistent pointers contained in objects in page P, using the extra information stored in the page. It takes the following actions for each persistent pointer in the page. (Let the value of the persistent pointer be $\langle p_i, o_i \rangle$, where p_i is the short page identifier and o_i is the offset within the page. Let P_i be the full page identifier of p_i, found in the translation table in page P.)

1. If page P_i does not already have a virtual-memory page allocated to it, the system now allocates a free page in virtual memory to it. The page P_i will reside at this virtual-memory location if and when it is brought in. At this point, the page in virtual address space does not have any storage allocated for it, either in memory or on disk; it is merely a range of addresses reserved for the database page. The system allocates actual space when it actually loads the database page P_i into virtual memory.

2. Let the virtual-memory page allocated (either earlier or in the preceding step) for P_i be v_i. The system updates the persistent pointer being considered, whose value is $\langle p_i, o_i \rangle$, by replacing p_i with v_i.

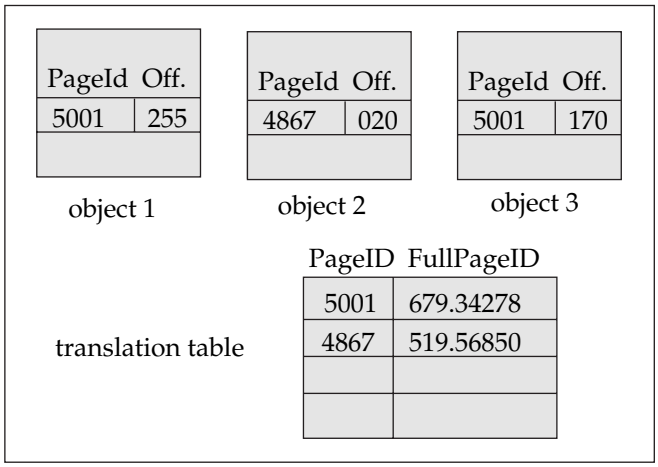


Figure 11.23 Page image after swizzling.

Figure 11.23 shows the state of the page from Figure 11.22 after the system has brought that page into memory and swizzled the pointers in it. Here, we assume that the page whose database page identifier is 679.34278 has been mapped to page 5001 in memory, whereas the page whose identifier is 519.56850 has been mapped to page 4867 (which is the same as the short page identifier). All the pointers in objects have been updated to reflect the new mapping, and can now be used as in-memory pointers.

At the end of the translation phase for a page, the objects in the page satisfy an important property: All persistent pointers contained in objects in the page have been converted to in-memory pointers. Thus, objects in in-memory pages contain only in-memory pointers. Routines that use these objects do not even need to know about the existence of persistent pointers! For example, existing libraries written for in-memory objects can be used unchanged for persistent objects. That is indeed an important advantage!

11.9.4.3 Pointer Dereference

Consider the first time that an in-memory pointer to a virtual-memory page v_i is dereferenced, when storage has not yet been allocated for the page. As we described, a segmentation violation will occur, and will result in a function call on the database system. The database system takes the following actions:

1. It first determines what database page was allocated to virtual-memory page v_i; let the full page identifier of the database page be P_i. (If no database page has been allocated to v_i, the pointer is incorrect, and the system flags an error.)
2. It allocates storage space for page v_i, and loads the database page P_i into virtual-memory page v_i.

3. It carries out pointer swizzling out on page P_i, as described earlier in "Swizzling Pointer on a Page".

4. After swizzling all persistent pointers in P, the system allows the pointer dereference that resulted in the segmentation violation to continue. The pointer dereference will find the object for which it was looking loaded in memory.

If any swizzled pointer that points to an object in page v_i is dereferenced later, the dereference proceeds just like any other virtual-memory access, with no extra overheads. In contrast, if swizzling is not used, there is considerable overhead in locating the buffer page containing the object and then accessing it. This overhead has to be incurred on *every* access to objects in the page, whereas when swizzling is performed, the overhead is incurred only on the *first* access to an object in the page. Later accesses operate at regular virtual-memory access speeds. Hardware swizzling thus gives excellent performance benefits to applications that repeatedly dereference pointers.

11.9.4.4 Optimizations

Software swizzling performs a deswizzling operation when a page in memory has to be written back to the database, to convert in-memory pointers back to persistent pointers. Hardware swizzling can avoid this step—when the system does pointer swizzling for the page, it simply updates the translation table for the page, so that the page-identifier part of the swizzled in-memory pointers can be used to look up the table. For example, as shown in Figure 11.23, database page 679.34278 (with short identifier 2395 in the page shown) is mapped to virtual-memory page 5001. At this point, not only is the pointer in object 1 updated from 2395255 to 5001255, but also the short identifier in the table is updated to 5001. Thus, the short identifier 5001 in object 1 and in the table match each other again. Therefore, the page can be written back to disk without any deswizzling.

Several optimizations can be carried out on the basic scheme described here. When the system swizzles page P, for each page P' referred to by any persistent pointer in P, it attempts to allocate P' to the virtual address location indicated by the short page identifier of P' on page P. If the system can allocate the page in this attempt, pointers to it do not need to be updated. In our swizzling example, page 519.56850 with short page identifier 4867 was mapped to virtual-memory page 4867, which is the same as its short page identifier. We can see that the pointer in object 2 to this page did not need to be changed during swizzling. If every page can be allocated to its appropriate location in virtual address space, none of the pointers need to be translated, and the cost of swizzling is reduced significantly.

Hardware swizzling works even if the database is bigger than virtual memory, but only as long as all the pages that a particular process accesses fit into the virtual memory of the process. If they do not, a page that has been brought into virtual memory will have to be replaced, and that replacement is hard to do, since there may be in-memory pointers to objects in that page.

Hardware swizzling can also be used at the level of sets of pages (often called segments), instead of for a single page. For set-level swizzling, the system uses a single translation table for all pages in the segment. It loads pages in the segment and swizzles them as and when they are required; they need not be loaded all together.

11.9.5 Disk Versus Memory Structure of Objects

The format in which objects are stored in memory may be different from the format in which they are stored on disk in the database. One reason may be the use of software swizzling, where the structures of persistent and in-memory pointers are different. Another reason may be that we want to have the database accessible from different machines, possibly based on different architectures, and from different languages, and from programs compiled under different compilers, all of which result in differences in the in-memory representation.

Consider, for example, a data-structure definition in a programming language such as C++. The physical structure (such as sizes and representation of integers) in the object depends on the machine on which the program is run.[5] Further, the physical structure may also depend on which compiler is used—in a language as complex as C++, different choices for translation from the high-level description to the physical structure are possible, and each compiler can make its own choice.

The solution to this problem is to make the physical representation of objects in the database independent of the machine and of the compiler. The system can convert the object from the disk representation to the form that is required on the specific machine, language, and compiler, when that object is brought into memory. It can do this conversion transparently at the same time that it swizzles pointers in the object, so the programmer does not need to worry about the conversion.

The first step in implementing such a scheme is to define a common language for describing the structure of objects—that is, a data-definition language. One such language is the Object Definition Language (ODL) developed by the Object Database Management Group (ODMG). ODL has mappings defined to the Java, C++, and Smalltalk languages, so potentially we may manipulate objects in an ODMG-compliant database using any of these languages.

The definition of the structure of each class in the database is stored (logically) in the databases. The code to translate an object in the database to the representation that is manipulated with the programming language (and vice versa) depends on the machine as well as on the compiler for the language. We can generate this code automatically, using the stored definition of the class of the object.

An unexpected source of differences between the disk and in-memory representations of data is the hidden-pointers in objects. **Hidden pointers** are transient pointers

5. For instance, the Motorola 680x0 architectures, the IBM 360 architecture, and the Intel 80386/80486/Pentium/Pentium-II/Pentium-III architectures all have 4-byte integers. However, they differ in how the bits of an integer are laid out within a word. In earlier-generation personal computers, integers were 2 bytes long; in newer workstation architectures, such as the Compaq Alpha, Intel Itanium, and Sun UltraSparc architectures, integers can be 8 bytes long.

that compilers generate and store in objects. These pointers point (indirectly) to tables used to implement certain methods of the object. The tables are typically compiled into executable object code, and their exact location depends on the executable object code; hence, they may be different for different processes. Therefore, when a process accesses an object, the hidden pointers must be fixed to point to the correct location. The hidden pointers can be initialized at the same time that data-representation conversions are carried out.

11.9.6 Large Objects

Objects may also be extremely large; for instance, multimedia objects may occupy several megabytes of space. Exceptionally large data items, such as video sequences, may run into gigabytes, although they are usually split into multiple objects, each on the order of a few megabytes or less. Large objects containing binary data are called binary large objects (blobs), while large objects containing character data, are called character large objects (clobs), as we saw in Section 9.2.1.

Most relational databases restrict the size of a record to be no larger than the size of a page, to simplify buffer management and free-space management. Large objects and long fields are often stored in a special file (or collection of files) reserved for long-field storage.

Allocation of buffer pages presents a problem with managing large objects. Large objects may need to be stored in a contiguous sequence of bytes when they are brought into memory; in that case, if an object is bigger than a page, contiguous pages of the buffer pool must be allocated to store it, which makes buffer management more difficult.

We often modify large objects by updating part of the object, or by inserting or deleting parts of the object, rather than by writing the entire object. If inserts and deletes need to be supported, we can handle large objects by using B-tree structures (which we study in Chapter 12). B-tree structures permit us to read the entire object, as well as to insert and delete parts of the object.

For practical reasons, we may manipulate large objects by using application programs, instead of doing so within the database:

- **Text data**. Text is usually treated as a byte string manipulated by editors and formatters.
- **Image/Graphical data**. Graphical data may be represented as a bitmap or as a set of lines, boxes, and other geometric objects. Although some graphical data often are managed within the database system itself, special application software is used for many cases, such as integrated circuit design.
- **Audio and video data**. Audio and video data are typically a digitized, compressed representation created and displayed by separate application software. Data are usually modified with special-purpose editing software, outside the database system.